

Proyecto de Investigación — Programación Lógica y Funcional 2026 “B”

Bloque 1: Programación Funcional — 40 temas de investigación y rúbrica de 5 categorías

TecNM Campus Tijuana — Ingeniería en Sistemas Computacionales (ISC-2006)

Agosto 2026

Presentación

Esta es la **primera lista de temas de investigación** del curso Programación Lógica y Funcional 2026 “B”. Cubre exclusivamente el **paradigma funcional** (Erlang/OTP, Elixir, Haskell, OCaml, Clojure, Scala, Gleam). Los temas de **programación lógica** (Prolog, Datalog, ASP, CLP(FD)) se publicarán en una lista posterior.

Cada estudiante desarrollará **una investigación técnica individual**, la publicará mediante el flujo **Fork — Pull Request** y documentará el uso de asistentes de IA (LLM).

Los 40 temas de esta lista fueron **revisados para no duplicarse** con las investigaciones ya integradas en **research/**:

- *Prolog y modelos de lenguaje grandes: aproximación neuro-simbólica* (paradigma lógico),
- *Erlang/OTP y LLMs: arquitectura tolerante a fallos* (queda excluido como tema).

No se admiten temas repetidos ni variantes menores de un tema ya trabajado; el docente asigna o confirma el tema por Google Classroom.

Entrega esperada

Carpeta personal dentro de **research/<nombre-del-tema>/** con:

- **README.md** — título, introducción, desarrollo técnico (mínimo 500 palabras), conclusiones y bibliografía en formato **IEEE**.
- **anexo.md** — bitácora de uso de LLM: prompts reales, resultados obtenidos y reflexión crítica (¿ayudó?, ¿hubo sesgos o errores?). Obligatorio si se usó IA.
- (*Optional*) código que compile/ejecute, diagramas y PDF de papers de referencia.

Reglas del flujo Fork — Pull Request

- No cambiar la ruta indicada ni renombrar **README.md**.
- No modificar archivos ajenos ni el **README.md** de otras carpetas.
- Un Pull Request por estudiante, con commits claros y descriptivos.
- El PR que altere la estructura del repositorio se **rechaza** sin calificación.

Reglas de contenido (según **CLAUDE.md** y **casos_reales_mundo_real.md**)

- Todo ejemplo de código debe compilar/ejecutar; el pseudocódigo se etiqueta como tal.
- Erlang: usar comportamientos OTP (**gen_server**, **supervisor**), nunca **spawn** sin supervisión.

- Haskell: `Maybe`/`Either` para errores, nunca funciones parciales sobre listas arbitrarias.
- Casos de industria: solo afirmaciones verificables con fuente (WhatsApp/Erlang, Discord/Elixir, Nubank/Clojure, Jane Street/OCaml, Standard Chartered/Haskell). No atribuir a organizaciones locales (IMSS, SAT, maquiladoras) un stack concreto sin fuente directa.

Los 40 temas — Bloque Funcional (2026 “B”)

Fundamentos y modelo de evaluación

1. Transparencia referencial y efectos secundarios: cómo razonan el compilador y el programador sobre código puro
2. Evaluación perezosa frente a estricta: *thunks*, `seq` y control de la fuerza en Haskell
3. Listas y estructuras infinitas: primos y Fibonacci con evaluación por necesidad
4. Memoización y programación dinámica en lenguajes puros sin estado mutable
5. *Space leaks* por pereza: diagnóstico y corrección con perfiles de memoria en GHC

Sistema de tipos y garantías en tiempo de compilación

6. Tipos algebraicos de datos y *pattern matching* exhaustivo como sustituto de `null`
7. *Type classes* en Haskell frente a *traits/protocols*: polimorfismo ad hoc resuelto en compilación
8. Inferencia de tipos Hindley–Milner: el algoritmo W y sus límites
9. El sistema de módulos de OCaml: funtores y firmas `.mli` (Flow e Infer de Meta)
10. `Maybe/Either` y manejo de errores sin excepciones: comparación con `try/catch`
11. Gleam: tipado estático sobre la BEAM y comparación con Erlang/Elixir dinámicos

Recursión, inmutabilidad y estructuras de datos

12. Optimización de llamada de cola (TCO): qué garantiza cada *runtime* (BEAM, GHC, JVM)
13. Trampolines y estilo de paso de continuaciones (CPS) para recursión profunda
14. Estructuras de datos persistentes: *Hash Array Mapped Trie* (HAMT) en Clojure
15. *Zippers*: navegación y actualización funcional de árboles
16. Costo real de la inmutabilidad: *structural sharing* y su medición

Funciones de orden superior y abstracción

17. Composición de funciones y estilo *point-free*: legibilidad frente a concisión
18. Functor, Applicative y Monad: la jerarquía y para qué sirve cada nivel
19. La mónada `State`: estado explícito sin variables mutables
20. Transformadores de mónadas y el problema de combinar efectos
21. Efectos algebraicos y *handlers*: la alternativa moderna a los transformadores
22. *Free monads* e intérpretes: separar la descripción de un programa de su ejecución
23. Lentes (*lenses*) y ópticas: actualización componible de estructuras anidadas

Concurrencia y modelo de actores

24. El planificador de la BEAM: procesos ligeros, *reductions* y expropiación
25. `gen_server` y `supervisor`: árboles de supervisión y la filosofía “let it crash”
26. *Backpressure* con GenStage y Flow en Elixir para flujos de datos
27. `core.async` en Clojure: canales y CSP sobre la JVM
28. Memoria transaccional por software (STM): `refs/dosync` en Clojure y `TVar` en Haskell
29. Recarga de código en caliente (*hot code reloading*) en Erlang/OTP

Procesamiento de datos y *streaming*

30. *Transducers* en Clojure: transformaciones componibles independientes de la colección
31. `Stream` perezoso para procesar archivos grandes sin cargarlos en memoria
32. Scala y Spark: el paradigma funcional aplicado a datos distribuidos
33. *Event sourcing* y CQRS: estado derivado de un registro inmutable de eventos

Metaprogramación y lenguajes de dominio específico

- 34. Macros higiénicas en Elixir: `quote/unquote` y `macroexpand`
- 35. Macros en Clojure y construcción de un DSL interno
- 36. Phoenix LiveView: un DSL funcional para interfaces web con estado en el servidor

Verificación, pruebas y corrección

- 37. *Property-based testing* con QuickCheck/PropEr: generar casos en vez de escribirlos
- 38. Dialyzer y *success typing*: análisis estático de discrepancias en código Erlang
- 39. Totalidad y funciones parciales: por qué `head/tail` son un riesgo
- 40. Introducción a la demostración asistida (Coq/Lean): los tipos como proposiciones

Rúbrica de evaluación — 5 categorías (100 puntos)

#	Categoría	Pts	Qué se evalúa
1	Rigor técnico y profundidad	30	Comprensión correcta y profunda del tema; exactitud de los conceptos; fuentes actualizadas y pertinentes; desarrollo técnico de al menos 500 palabras con datos, comparativas o ejemplos que compilan/ejecutan.
2	Estructura y claridad del README.md	20	Uso correcto de Markdown; organización lógica (introducción, desarrollo, conclusiones); redacción profesional y ortografía sin faltas graves.
3	Originalidad y análisis crítico	20	Síntesis y redacción propias; el texto no es una copia de la salida de un LLM; hay interpretación, comparación entre lenguajes o paradigmas y postura argumentada del estudiante.
4	Uso del repositorio y flujo Fork — Pull Request	15	Ruta y nombre de carpeta correctos; README.md sin renombrar; no se tocan archivos ajenos; commits claros; un solo PR bien descrito.
5	Bitácora de IA (anexo.md) y bibliografía IEEE	15	anexo.md con prompts reales, resultados y reflexión honesta sobre el uso de IA; bibliografía con fuentes confiables (IEEE, libros, papers, sitios oficiales) y formato IEEE correcto.

Escala de desempeño por categoría

Nivel	Porcentaje de la categoría	Descripción
Excelente	90–100 %	Cumple todos los criterios con evidencia sólida y sin observaciones.
Satisfactorio	75–89 %	Cumple lo esencial con observaciones menores.
Suficiente	60–74 %	Cumple parcialmente; faltan elementos o hay imprecisiones.
Insuficiente	0–59 %	No cumple el criterio mínimo o hay copia sin análisis.

Penalizaciones

- **Tema duplicado** o variante menor de una investigación ya integrada en **research/**: se devuelve el PR sin calificar hasta reasignar tema.

- **Afirmación de industria sin fuente verificable** (atribuir un stack FP a una organización sin cita directa): categoría 1 con penalización según gravedad.
- **Código que no compila/ejecuta** presentado como funcional: penalización en categoría 1.
- **Alteración de la estructura del repositorio** o de archivos ajenos: PR rechazado (categoría 4 en 0).
- **Uso de IA no declarado** detectado: categorías 3 y 5 en 0 y reporte de integridad académica.
- **Entrega tardía**: según las políticas del curso.

Documento del curso Programación Lógica y Funcional (ISC-2006), semestre 2026 “B”. Bloque 1 de 2: Programación Funcional. El bloque de Programación Lógica se publicará por separado. Referencias del curso: SYLLABUS.md, README.md, casos_reales_mundo_real.md, CLAUDE.md.